Руководство по Embassy на русском

Почта: denj.n3.0@gmail.com
Github: https://github.com/Nishekuro/ru-embassy-tutorial

Содержание

1 Введение 3
1.1 Что такое Embassy 3
1.2 Настройка рабочего окружения 3
1.3 Настройка компилятора под микроконтроллер 4
1.4 Прошивка и отладка через probe-rs 7
2 Структура проекта 9
2.1 Создание минимального проекта 9
2.1.1 .cargo.config.toml 9
2.1.2 Cargo.toml 10
2.1.3 build.rs 11
2.1.4 src/main.rs 12
2.1.5 Настройка Zed 12
2.1.6 Итоговая структура проекта 13
2.2 Blink 13
2.3 Утилиты создания проекта 14
3 GPIO и EXTI 15

1 Введение

Прежде чем приступить к изучению данного руководства необходимо ознакомиться с основами языка Rust.

- Официальный учебник Rust
- Официальный перевод учебника

Также необходимо установить или убедиться, что базовый пакет языка Rust уже установлен в вашей системе:

```
rustc --version
```

Для понимания специфики embedded-разработки на Rust также рекомендуется ознакомиться с первыми тремя главами The Embedded Rust Book. Однако данное руководство минует HAL уровень разработки, и некоторые необходимые идеи из вышеупомянутой книги даны здесь.

Руководство находится на стадии активной разработки. Если вы заметили ошибку, неточность, хотите предложить улучшение или задать вопрос, свяжитесь со мной —.

1.1 Что такое Embassy

Embassy — это фреймворк для embedded-разработки на Rust, построенный вокруг концепции асинхронности.

В классическом embedded-коде периферией управляют через прерывания и колбэки, что приводит к сложной логике и состояниям. Embassy предлагает **асинхронные задачи** (async tasks), которые:

- ожидают события (нажатие кнопки, приём данных по UART, срабатывание таймера) без блокировки процессора;
- не требуют RTOS — встроенный executor эффективно переключается между задачами;
- используют преимущества Rust: безопасность памяти, отсутствие гонок данных, удобные абстракции.

Embassy включает:

- аппаратные драйверы для многих чипов (STM32, RP2040, ESP32, nRF);
- утилиты для работы с временем (`embassy-time`), синхронизации (`embassy-sync`), USB, сетью и другими протоколами;
- инструменты для эффективного логирования (`defmt`) и отладки.

Этот подход позволяет писать читаемый, поддерживаемый и безопасный embedded-код, не уступающий по производительности традиционным решениям.

1.2 Настройка рабочего окружения

В современных условиях использование Visual Studio Code на территории РФ сопряжено с проблемами доступа к встроенному магазину плагинов,

решение которых создает дополнительный порог входа, особенно для начинающих. Также стоит обратить внимание на то, что официальная сборка VSCode содержит проприетарные компоненты, а сам редактор построен на Electron, что сказывается на производительности.

В этом руководстве рекомендуется использовать **Zed**. Редактор написан на Rust, обеспечивает высокую скорость работы, не имеет подобных проблем со скачиванием плагинов, а многие инструменты разработки (как rust-analyzer) либо подключены "из коробки", либо подхватываются автоматически. Интерфейс Zed минималистичен и позволяет сосредоточиться на коде.

Основные недостатки Zed связаны с молодостью проекта: настройка через JSON-файлы вместо графического интерфейса, скромная экосистема расширений и отсутствие встроенного GUI-отладчика. Для отладки потребуется терминал и probe-rs, что соответствует традиционному подходу в embedded-разработке.

Выбор Zed продиктован доступностью и предсказуемостью: редактор не зависит от внешних сервисов и обеспечивает необходимый минимум функциональности. Если вы предпочитаете другие инструменты — это допустимо, но руководство ориентируется на Zed, а все примеры приведены с расчетом на работу в терминале.

Самостоятельно ознакомиться с редактором и скачать актуальную версию для вашей операционной системы можно на официальном сайте Zed.

Таблица 1 – Основные сочетания клавиш Zed

Команда	Сочетание клавиш
Создать терминал	Ctrl + ~
Создать дополнительный терминал	Ctrl + Shift + ~
Переключиться между терминалами	Ctrl + Tab
Закрыть терминал	Ctrl + Shift + w
Очистить терминал	Ctrl + Shift + l
Переключение комментирования строки	Ctrl + /
…	…

1.3 Настройка компилятора под микроконтроллер

В классическом случае разработка на языке Rust ведется для desktop-архитектур, таких как x86 или ARM. В минимальной конфигурации rustup уже включает все необходимые инструменты для компиляции кода под вашу систему. Однако архитектура микроконтроллеров кардинально отличается — там нет операционной системы, а система команд процессора и организация памяти сильно разнится от платформы к платформе.

Target — это описание аппаратной платформы, для которой компилируется код. Он сообщает компилятору:

- какую систему команд использовать (какие инструкции процессора доступны);
- есть ли операционная система (или это «голое железо» — bare-metal);
- как организована память (разрядность указателей, выравнивание);
- какие соглашения о вызовах функций применять.

Процесс, при котором код компилируется на одной платформе, а выполняется на другой, называется кросс-компиляцией.

Все target'ы в Rust делятся на три уровня поддержки. Это не оценка качества, а показатель того, насколько автоматизировано тестирование платформы:

Таблица 2 – Уровни поддержки платформ

Уровень	Описание	Примеры
Tier 1	Полностью поддерживаются, проходят все тесты в CI, есть официальные сборки. "Гарантированно работает"	`x86_64-unknown-linux-gnu`, `aarch64-apple-darwin`
Tier 2	Гарантированно собираются, есть официальные сборки, но тесты могут не проходить автоматически	`aarch64-unknown-none`, `arm-unknown-linux-gnueabi`
Tier 3	Поддержка есть, но без автоматического тестирования в CI. Код компилируется, но может сломаться при обновлении компилятора	Большинство embedded-платформ: STM32, RP2040, ESP32 и пр.

Не стоит пугаться tier 3 для embedded-платформ. Это нормальная ситуация, связанная с особенностями bare-metal-разработки:

- для bare-metal нет абстракции "виртуальной платы" и единого интерфейса – у каждого чипа своя память, регистры, способ прошивки;
- только семейство STM32 насчитывает сотни различных чипов – добавить каждый в систему непрерывной интеграции (CI) физически невозможно;
- для прошивки и отладки нужны реальные программаторы (ST-Link, J-Link, USB-UART) и сами платы – автоматизировать их подключение в облачном CI — сложная и дорогостоящая задача;

- каждая платформа требует своих утилит: для STM32 — ST-Link, для ESP32 — esptool, для RP2040 — picotool – поддерживать их все в единой инфраструктуре тестирования крайне сложно.

Для автоматического тестирования embedded-проектов обычно используется QEMU-эмулятор (подход описан в соответствующей главе The Embedded Rust Book). Однако QEMU не может гарантировать полной совместимости с реальным чипом. Он эмулирует поведение, но тайминги, прерывания и специфическая периферия могут работать иначе. Финальное тестирование обязательно должно проходить на реальном железе.

Теперь, когда мы понимаем, что такое target и зачем он нужен, разберемся, как подобрать нужный. Со списком всех поддерживаемых на данный момент платформ можно ознакомиться в соответствующем разделе The rustc book. К сожалению, алгоритм определения необходимого target`а специфичен для каждой платформы. В общем случае он выглядит следующим образом:

- определить маркировку чипа (например, STM32F411CE, RP2040, ESP32-C3);
- определить архитектуру чипа (Cortex-M0+, Cortex-M4 с FPU, RISC-V и т.д.);
- соотнести архитектуру чипа с официальным списком.

Вот пример для популярных платформ:

Платформа	Ядро	Архитектура	Target
STM32F103	Cortex-M3	ARMv7-M	`thumbv7m-none-eabi`
STM32F411	Cortex-M4	ARMv7E-M	`thumbv7em-none-eabihf`
RP2040	Cortex-M0+	ARMv6-M	`thumbv6m-none-eabi`
ESP32-C3	RISC-V	RISC-V	`riscv32imc-unknown-none-elf`

После того как определились с архитектурой, добавим соответствующий target в rustup консольной командой `rustup target add`. Далее представлен пример добавления target`ов для популярных платформ. Откройте терминал (PowerShell на Windows или консоль в Zed) и введите одну из следующих команд:

```
rustup target add thumbv7em-none-eabihf       # STM32F411CE
(black pill)
rustup target add thumbv7m-none-eabi          # STM32F103C8
(blue pill)
rustup target add thumbv6m-none-eabi          # RP2040
(Raspberry Pi Pico)
rustup target add riscv32imc-unknown-none-elf # ESP32-C3
```

Убедиться, что платформа была добавлена и узнать, какие target`ы сейчас установлены в системе можно следующей командой:

```
rustup target list --installed
```

Далее большинство примеров в данном руководстве будут обозреваться на базе микроконтроллера STM32F411CE.

1.4 Прошивка и отладка через probe-rs

Rrobe-rs – это современный инструмент для работы с embedded-устройствами, уже ставший стандартом для Rust разработчиков. Он выполняет такие функции, как:

- прошивка микроконтроллера;
- отладка с пошаговым выполнением (точки останова);
- передача данных без остановки кода (Real-Time Transfer);
- эффективное логирование с помощью defmt.

В отличие от проприетарных решений (ST-Link Utility, J-Link Commander), probe-rs:

- работает с десятками программаторов (ST-Link, J-Link, CMSIS-DAP, FTDI и др.)
- не требует лицензий и работает на всех ОС
- интегрируется напрямую с Cargo через `cargo run`

Так как Rust уже есть в нашей системе, воспользуемся наиболее универсальным способом установки probe-rs через консольную команду `cargo binstall`:

```
cargo binstall probe-rs-tools
```

Когда probe-rs установлен, следует проверить, видит ли он наш программатор (либо саму плату, если программатор не требуется). Подключите устройство к вашему компьютеру и введите в консоль команду `probe-rs list`. В случае с STM32F411, подключенным через программатор ST-Link/v2 при распознавании устройства мы получим сообщение примерно следующего содержания:

```
$ probe-rs list
The following debug probes were found:
[0]: STLink V2 -- 0483:3748:6068C3BF286068C3BF286068C3BF28
(ST-LINK)
```

На Windows многие USB-программаторы (ST-Link, J-Link, CMSIS-DAP) не распознаются автоматически и требуют установки драйверов. Это связано с тем, что Windows использует свои драйверы (например, WinUSB или libusb), а не стандартные USB-классы.

Zadig — это универсальная утилита для установки драйверов USB-устройств на Windows. Она работает с любым оборудованием и не требует поиска специфичных драйверов от производителя. Вот краткая инструкция по работе с Zadig:

- скачайте Zadig с https://zadig.akeo.ie/;
- подключите устройство к компьютеру;
- запустите Zadig от имени администратора;
- Откройте Options → List All Devices;
- выберите ваше устройство (ST-Link);
- выберите драйвер (WinUSB);
- нажмите "Install Driver".

Подробнее про первичную настройку probe-rs для каждой операционной системы написано на соответствующей странице Probe-rs.

Подробнее о работе с probe-rs будет написано далее по ходу учебника и в отдельной главе.

2 Структура проекта

- Официальное руководство Embassy
- Официальный репозиторий Embassy

2.1 Создание минимального проекта

Разберем специфику embedded rust на Embassy, создав минимальный проект вручную. Открыв терминал, создадим новый проект с названием `bilnky` классической командой:

```
cargo new blinky
```

Внутри мы увидим базовую структуру проекта:

```
blinky/
├── src/
│   └── main.rs
└── Cargo.toml
```

При открытии проекта в Zed, в корень будут добавлены объекты `target` и `Cargo.lock`. Таким образом, структура проекта будет выглядеть следующим образом:

```
blinky/
├── src/
│   └── main.rs
├── src/
├── Cargo.lock
└── Cargo.toml
```

2.1.1 .cargo.config.toml

Для того чтобы прошивать плату с помощью `probe-rs` в корне проекта необходимо создать файл `.cargo/config.toml` и прописать в него примерно следующую информацию:

```
[target.'cfg(all(target_arch = "arm", target_os =
"none"))']
runner = "probe-rs run --chip STM32F411CE"
[build]
target = "thumbv7em-none-eabihf"

[env]
DEFMT_LOG = "info"
```

Ключ `runner` позволяет нам удобно прошивать плату через `probe-rs` используя команду `cargo run`. Для этого необходимо прописать соответсвующий идентификатор чипа. Правильное написание идентификатора можно узнать использовав команду:

```
probe-rs chip list | grep -i "примерное_название_чипа"
```

Пример:

```
probe-rs chip list | grep -i stm32f411
        STM32F411CC
        STM32F411CE # <- точный идентификатор нашего чипа
        STM32F411RC
        STM32F411RE
        STM32F411VC
        STM32F411VE
```

В ключ `target` прописывается идентификатор архитектуры, который мы определили ранее.

2.1.2 Cargo.toml

Теперь, когда компилятор знает идентификатор чипа, добавим в проект все необходимые зависимости. Важно понимать, что, в силу своего недавнего появления, Embassy – очень нестабильный проект. Crates.io – площадка эсперементов данного проекта, и установка зависимостей командой стандартной командой `cargo add` крайне не рекомендуется; в большинстве случаем крейты не будут работать друг с другом. Гарантию работы можно получить в официальном репозитории на Github. Внутри директории `examples/` мы можем найти интересующую нас платформу, конфигурационный файл и примеры для нее. Такой конфигурационный файл содержит все крейты проекта наиболее свежих крейтов, прошедших тестирование на взаимную работу. Скопируем соответствующий файл `Cargo.toml` в корень своего проекта. Внутри мы должны увидеть примерно следующее:

```
...
[dependencies]
...
embassy-stm32 = { version = "0.6.0", features = [
  "defmt",
  "stm32f411ce",
  "unstable-pac",
  "memory-x",
  "time-driver-any",
  "exti",
```

```
] }
embassy-executor = { version = "0.10.0", features = [
  "platform-cortex-m",
  "executor-thread",
  "defmt",
] }
embassy-time = { version = "0.5.1", features = [
  "defmt",
  "defmt-timestamp-uptime",
] }

defmt = "1.0.1"
defmt-rtt = "1.0.0"

cortex-m = { version = "0.7.6", features = ["critical-
section-single-core"] }
cortex-m-rt = "0.7.0"

panic-probe = { version = "1.0.0", features = ["print-
defmt"] }
...
```

Для крейтов с названием `embassy-*` очень важным явдляется определить совместимую версию. Остальные крейты работают почти всегда.

Описание наиболее популярных крейтов будет дано далее по ходу руководства и в отдельной главе.

Чтобы в `release` сборе проекта отладочная информация сохранилась, необходимо это явно указать в `Cargo.toml`:

```
...
[profile.release]
debug = 2 # 2 - полная отладочная информация
...
```

2.1.3 build.rs

Данный файл является платформо-зависиммым, потому его всегда копируют из официального репозитория. Для stm32f411ce содержимое данного файла выглядит следующим образом:

```
fn main() {
    println!("cargo:rustc-link-arg-bins=--nmagic");
    println!("cargo:rustc-link-arg-bins=-Tlink.x");
    println!("cargo:rustc-link-arg-bins=-Tdefmt.x");
}
```

2.1.4 src/main.rs

Первое, что мы должны сделать перейдя в `src/main.rs`, это добавить следующие строки в самом начале проекта:

```
#![no_std]
#![no_main]

use {defmt_rtt as _, panic_probe as _};
```

`#![no_std]` указывает на то, что данная программа не будет использовать весь стандартный крейт (пакет) `std` полностью. Вместо этого программа будет работать только с его подмножеством - крейтом `core`. Этот крейт содержит лишь те компоненты, которые не зависят от операционной системы: не требуют выделения памяти, не работают со стандартными потоками и т.д.

`!#[no_main]` означает, что программа не будет использовать стандартный интерфейс `main`, так как она запускается вне операционной системы.

Следующая строка определяет то, как мы будем работать с паникой и обработкой ошибок. В процессе разработки хорошей практикой является опора на `defmt-rtt` и `panic-prode` для вывода диагностической информации в консоль.

2.1.5 Настройка Zed

Как говорилось ранее, редактор Zed (как и VSCode) в контексте embedded rust требует специфичной настройки каждого проекта через JSON-файлы. Мнинимальные действия, которые необходимо сделать, это дать LSP (rust-analyzer в нашем случае) понять, что работа в данном проекте ведется не для классической desctop архитектуры. Сделать это можно, создав в корне проекта скрытую директорию `.zed` (точка в начале названия указывает на то, что директория скрытая), и поместив внутрь файл `settings.json` со следующим содержимым:

```
{
  "lsp": {
    "rust-analyzer": {
      "initialization_options": {
        "cargo": {
          "target": "thumbv7em-none-eabihf",
        },
        "check": {
          "allTargets": false,
        },
      },
    },
```

```
  },
}
```

Сразу после сохранения файла, Zed должен сделать перепроверку всего проекта и перестать ругаться на первые строки файла `src/main.rs`.

2.1.6 Итоговая структура проекта

Структура файлов верхнего уровня вашего проекта должна выглядеть следующим образом:

```
blinky/
├── .zed/
├── └── settings.json
├── .cargo/
├── └── config.toml
├── src/
├── └── main.rs
├── build.rs
├── Cargo.toml
├── Cargo.lock
├── target/
├── {memory.x}
└── {rust-toolchain.toml}
```

Файлы, указанные внутри фигурных скобок могут потребоваться для работы с некоторыми микроконтроллерами (например RP2040), однако для работы с серией STM32F4 они не нужны.

2.2 Blink

Исходя из всего вышесказанного создадим свой первый рабочий проект, в котором просто помигаем встроенным светодиодом и выведем сообщения на экран. В файл `src/main.rs` добавим код, чтобы получилось следующее:

```
#![no_std]
#![no_main]

use defmt::*;                                    //
отладочная информация
use embassy_executor::Spawner;                   // базовый
компонент для работы фрейймворка embassy
use embassy_stm32::gpio::{Level, Output, Speed}; //
управление пинами входа-выхода
use embassy_time::Timer;                         // функции
```

```
ожидания
use {defmt_rtt as _, panic_probe as _};          //
обработка ошибок

#[embassy_executor::main]
async fn main(_spawner: Spawner) {
    let p = embassy_stm32::init(Default::default()); //
инициализацция переферии
    info!("Hello World!");

    let mut led = Output::new(p.PC13, Level::High,
Speed::Low); // инициализация пина C13

    loop {
        info!("high");
        led.set_low();
        Timer::after_secs(1).await;

        info!("low");
        led.set_high();
        Timer::after_secs(1).await;
    }
}
```

2.3 Утилиты создания проекта

`cargo generate` — это мощный инструмент для создания новых проектов из готовых шаблонов (templates). Для Embassy это особенно полезно, так как позволяет быстро получить правильно настроенную структуру проекта без ручного копирования конфигураций. Однако такой способ может проигрывать ручному, когда дело касается свежести крейтов, так как создатели таких утилит и шаблонов не всегда успевают за репозиторием Embassy.

На практике все теоретические преимущества автоматической генерации проекта на данный момент ликвидируются тем факом, что `Cargo.toml` подтягиваются не все возможные для данной платформы, а необходимый минимум для запуска hellow-world-программы.

```
Короче говоря, пока руками собирайте)))
```

3 GPIO и EXTI

Адаптируем код с сайта Embassy для STM32F411CE. Создадим новый проект с названием `start`. В `Cargo.toml` необходимо добавить поддержку EXTI:

```
...
embassy-stm32 = { version = "0.6.0", features = [
  "defmt",
  "stm32f411ce",
  "unstable-pac",
  "memory-x",
  "time-driver-any",
  "exti", # добавить эту строку
] }
...
```

В `src/main` перепишем адаптированный код:

```
#![no_std]
#![no_main]

use defmt::*; // отладочная информация
use embassy_executor::Spawner; // базовый компонент для
работы фрейймворка embassy
use embassy_stm32::{
    Peri, bind_interrupts,
    exti::{self, ExtiInput},
    gpio::{AnyPin, Level, Output, Pull, Speed},
    interrupt,
}; // управление пинами входа-выхода
use embassy_time::Timer; // функции ожидания
use {defmt_rtt as _, panic_probe as _}; // обработка ошибок

bind_interrupts!(
    pub struct Irqs{
        EXTI0 =>
exti::InterruptHandler<interrupt::typelevel::EXTI0>;
});

#[embassy_executor::task]
async fn blinky(pin: Peri<'static, AnyPin>) {
    let mut led = Output::new(pin, Level::High,
Speed::Low); // инициализация пина C13

    loop {
        info!("high");
        led.set_low();
```

```
        Timer::after_secs(1).await;

        info!("low");
        led.set_high();
        Timer::after_secs(1).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    let p = embassy_stm32::init(Default::default()); //
инициализацция переферии
    info!("Hello World!");

    spawner.spawn(blinky(p.PC13.into()).unwrap());

    let mut button = ExtiInput::new(p.PA0, p.EXTI0,
Pull::Up, Irqs);

    loop {
        button.wait_for_falling_edge().await;
        info!("Pressed!");
        button.wait_for_rising_edge().await;
        info!("Released!");
    }
}
```